RUB

RUHR-UNIVERSITÄT BOCHUM

ASSEMBLY

Computer Architecture

Agenda

- More control flow
- Memory access
- Instruction set design

More Control Flow

Recap: Jumps and Branches

```
j     imm                (Unconditional jump)
beq   rs1, rs2, imm      (Branch Equal)
bne   rs1, rs2, imm      (Branch Not Equal)
blt   rs1, rs2, imm      (Branch Less Than)
bge   rs1, rs2, imm      (Branch Greater or Equal)
bltu  rs1, rs2, imm      (Branch LT Unsigned)
bgeu  rs1, rs2, imm      (Branch GE Unsigned)
```

(Conditionally) sets PC to PC+`imm`

For example, `bne` branches if `rs1` ≠ `rs2`

Example – Triangular Numbers

$$\sum_{i=1}^{n} i = 1 + 2 + 3 + \cdots + n$$

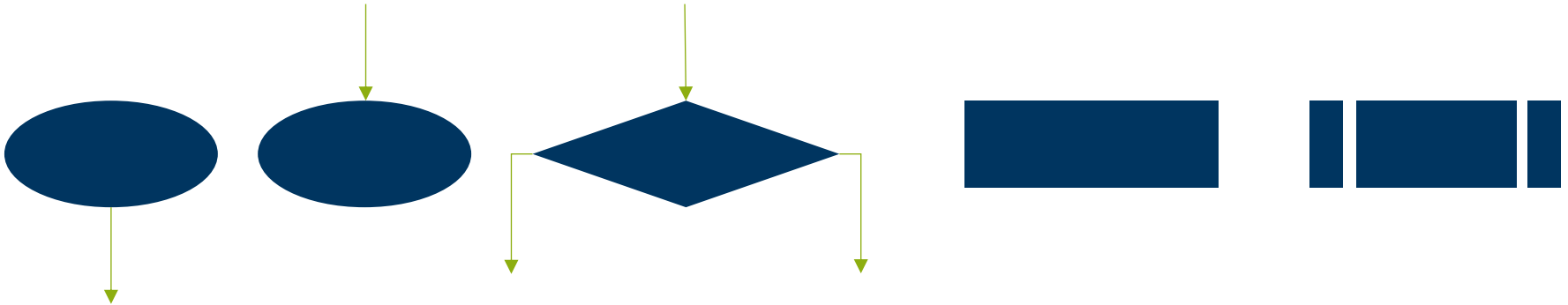
Increment a counter (i) until it reaches n

Add each counter value to the previous sum

Flow Charts

Graphic way to describe the control flow of a program

Elements:



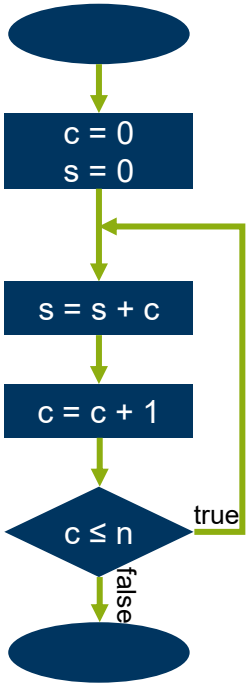
Entry Point | Exit Point | Condition | Instruction | Function Call

Example – Triangular Numbers

$$\sum_{i=1}^{n} i = 1 + 2 + 3 + \cdots + n$$

Increment a counter (i) until it reaches n

Add each counter value to the previous sum




```
    li   t0, 10

    li   t1, 0
    li   t2, 0
loop:
    add  t2, t2, t1

    addi t1, t1, 1

    bgeu t0, t1, loop

    nop
```

Example – Triangular Numbers – Gauss Summation

$$\sum_{i=1}^{n} i = 1 + 2 + 3 + \cdots + n = \frac{n \cdot (n+1)}{2}$$

```
    li   t0, 10
    li   t1, 0
    li   t2, 0
loop:
    add  t2, t2, t1
    addi t1, t1, 1
    bgeu t0, t1, loop
    nop
```

```
li   t0, 10
addi t1, t0, 1
mul  t0, t0, t1
srli t0, t0, 1
```

Do we always get the same result?

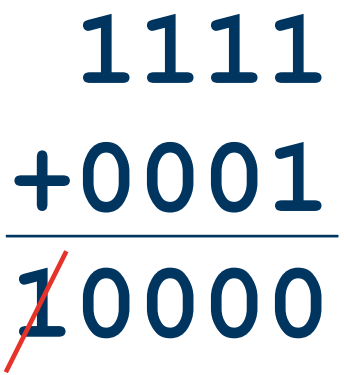
Handling Overflows

Overflow

When computing, e.g. a sum, and the result does not fit in a register, we lose information.

In RISC V there is no notification of such an overflow

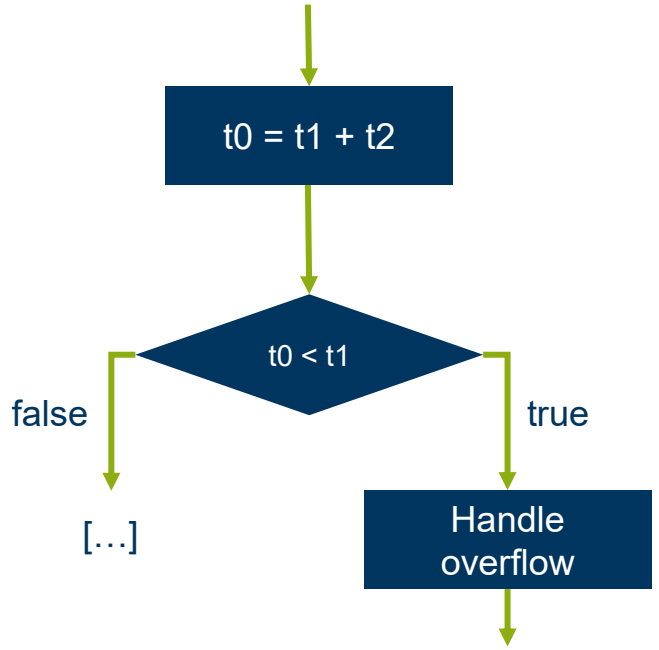
We need a way of detecting this.



Overflow – Unsigned Addition

Recall: all bits are used for the number

The result is expected to always be not less than the summands




```
bltu t0, t1, overflow
```

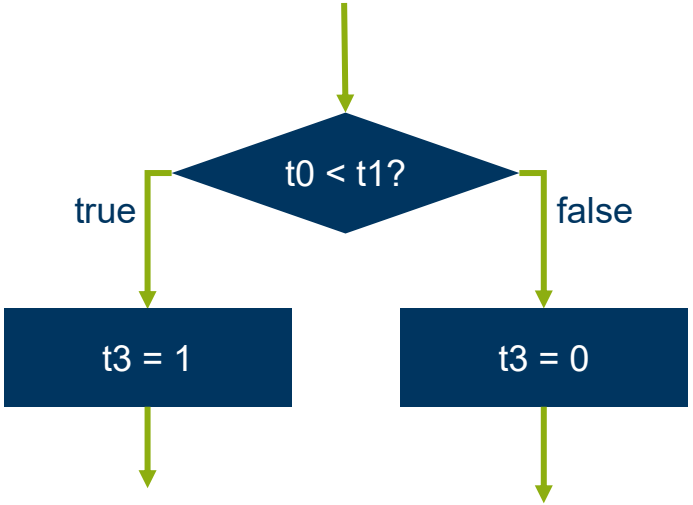
Getting the Carry

In many cases, we just need to compute the carry.

Using branches, we can do:

```
    add t0, t1, t2
    bltu t0, t1, overflow
    li t3, 0
    j continue
overflow:
    li t3, 1
continue:
```

But that's ugly…

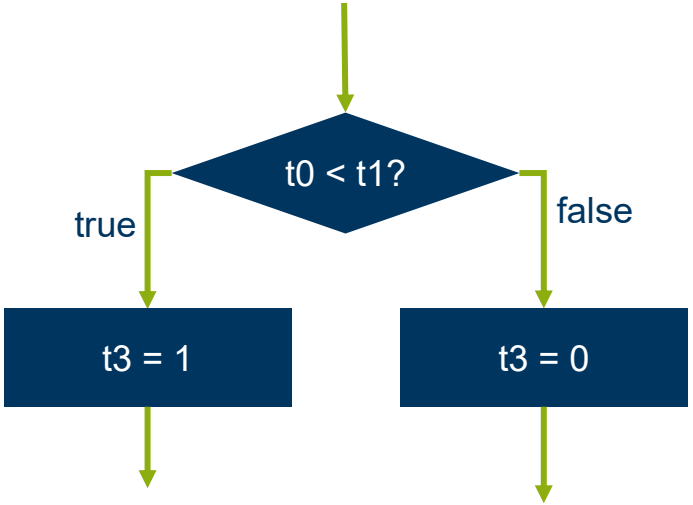


Comparisons

Check if the value of a register is less than that of another register/ immediate and save the result in a register

Assembly instructions:

- **S**et **L**ess **T**han
- **S**et **L**ess **T**han **I**mmediate
- **S**et **L**ess **T**han **U**nsigned
- **S**et **L**ess **T**han **U**nsigned **I**mmediate




```
sltu t3, t0, t1
```

Class exercise

Show three different instructions that compute:

$$\texttt{t1} = \begin{cases} 0 & \texttt{t0} \geq 0 \\ 1 & \texttt{t0} < 0 \end{cases}$$

Overflow – Signed Addition

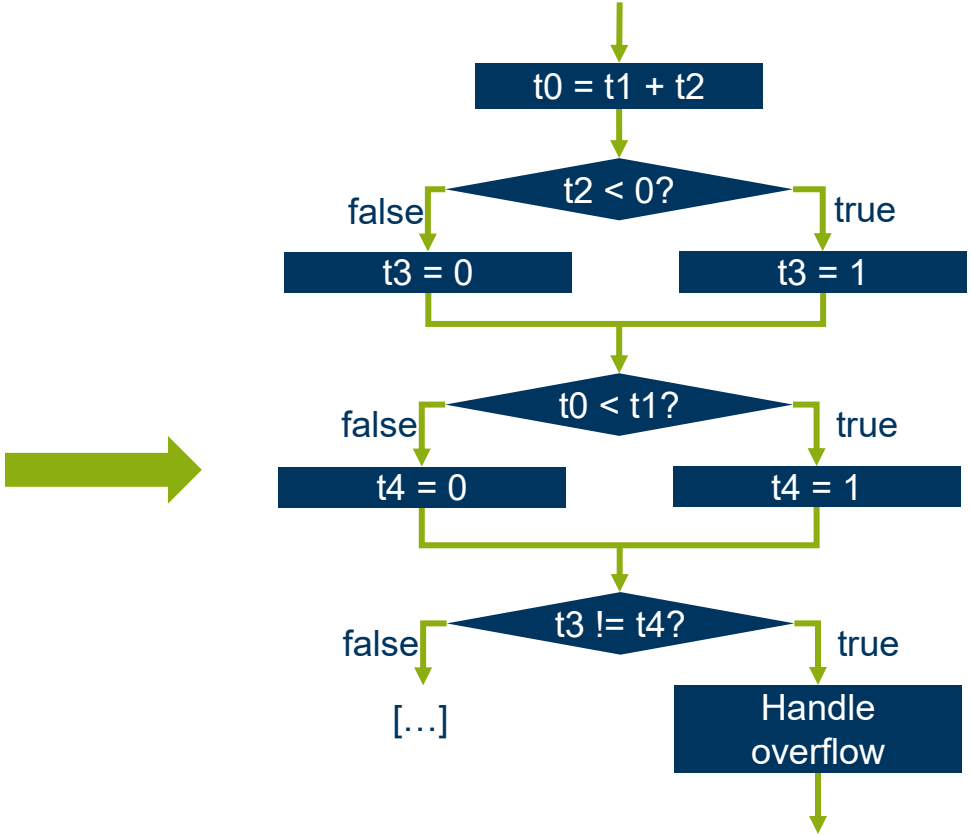
Reminder: the most significant bit is used for the sign

Operand 1	Operand 2	Sum
+	+	greater than or equal to both operands
+	-	less than operand 1, greater or equal to operand 2
-	+	greater or equal to operand 1, less than operand 2
-	-	less than both operands

The sum should be less than one operand only if the other operand is negative

Overflow Example

```
add  t0, t1, t2
slti t3, t2, 0
slt  t4, t0, t1
bne  t3, t4, overflow
```



Memory Access

Memory

- A sequence of bytes
  - Each byte is 8 bits

- Two (main) orders for storing longer words
  - Little endian: least significant byte first
  - Big endian: most significant byte first

Counting Bytes

8 bits = 1 byte

Name	Symbol	Number of bytes
Byte	B	10^0
Kilobyte	KB	10^3
Megabyte	MB	10^6
Gigabyte	GB	10^9
Terrabyte	TB	10^{12}

Name	Symbol	Number of bytes
Byte	B	2^0
Kibibyte	KiB	2^{10}
Mebibyte	MiB	2^{20}
Gibibyte	GiB	2^{30}
Tebibyte	TiB	2^{40}

LOAD/STORE Architecture

LOAD Instructions: Load data from memory into a register

STORE Instructions: Store data from a register in memory

All other instructions operate on registers only

Absolute Addressing

Address is specified at the time of programming

```
Result = Mem[Addr]
```

Example (Addr = 2):

```
Result = Mem[2]
       = 4
```

Address	Value (Example)
1	253
2	4
3	235
4	25
5	134
6	124
7	62

Register Offset Addressing

Address is calculated by adding the offset to the value of a given register.

```
Result =
Mem[Value(Rs1 + Offset]
```

Example

(Value Rs1 = 3, Offset = -1):

```
Result = Mem[3 + (-1)]
       = Mem[2]
       = 4
```

Address	Value (Example)
1	253
2	4
3	235
4	25
5	134
6	124
7	62

PC-Relative Addressing

Address is calculated by adding an offset to the PC value of the next instruction

```
Result = Mem[PC+Offset]
```

Example (Offset = 1, PC = 0):

```
Result = Mem[1+4]
       = Mem[5]
       = 134
```

Address	Value (Example)
1	253
2	4
3	235
4	25
5	134
6	124
7	62

Loads from Memory

Instructions to load data from memory into a register

```
lw rd, offset(rs1)
```

Emulating other addressing modes

Use `lui` or `x0` for absolute addresses

```
lw t0, 160(x0)
```

```
lui t0, 0xf4
lw t0, 0x240(t0)
```

Use `auipc` or `lw` without register for PC relative addressing

```
lw t0, data
```

```
auipc t0, 0xf4
lw    t0, 0x240(t0)
```

Assembly Instructions

- **L**oad **B**yte
- **L**oad **H**alf-Word
- **L**oad **W**ord
- **L**oad **B**yte (**U**nsigned)
- **L**oad **H**alf-Word (**U**nsigned)

Byte: 8 bit
Half-Word: 16 bit
Word: 32 bit

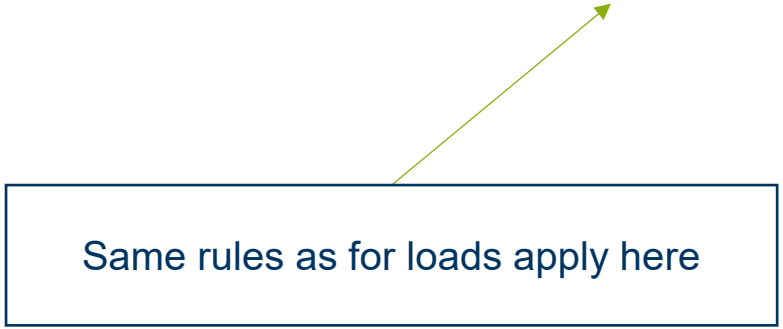
zero or sign extension is done for all loads

All Instructions set the least significant x bits

Stores

Instructions to write data from a register into memory

```
sw rs2, offset(rs1)
```



Assembly Instructions

- **S**tore **B**yte
- **S**tore **H**alf-Word
- **S**tore **W**ord

All instructions store the least significant x bits

Example

Code:

```
li t0, 5
sw t0, 12(x0)
lw t2, 12(x0)
```

Register:

x5 (t0)	0x00000005
x7 (t2)	0x00000000

Memory:

Address	Word
0x00000000	0xbaadf00d
0x00000004	0xdeadbeef
0x00000008	0xf0cacc1a
0x0000000c	0x00000005

RUHR UNIVERSITÄT BOCHUM RUB

More Control Flow

Assembly Instruction: Jump and Link

```
jal rd, imm
```

Jump to `pc+imm` and save the return address in `rd`

Standard calling convention uses `ra` (`x1`) for return address

In practice, `j` is a pseudo instruction –
uses register `zero` (`x0`) as a link register

Jump and Link Register

```
jalr rd, rs, imm
```

Jump to `rs+imm` and save the return address in `rd`

```
ret          ➡  jalr  x0, ra, 0

call label   ➡  auipc ra, (label-pc+2048)>>12
                jalr  ra, ra, (label-pc-4)&0xfff
```

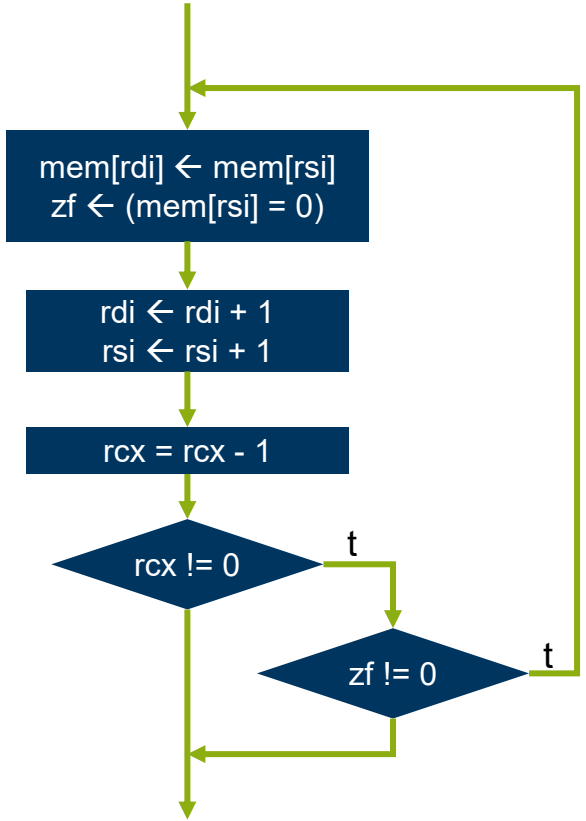
Instruction Set Design

Instruction Set Architecture

The set of instructions that a processor supports

Allow efficient implementation of common programming constructs

Tradeoff between hardware and software complexity



The x86 **`REPNZ MOVSB`** instruction

RISC vs.CISC

RISC	CISC
Reduced Instruction Set Computer	Complex Instruction Set Computer
Simple hardware, complex software	Simple software complex hardware

Early Computers CISC: complex and hard to program
Since the 1970s: RISC available, less complex instructions

Layers

Higher program languages

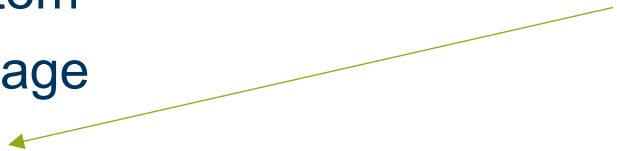
Assembler

Operating System

Machine language

Microprogram

Digital Level



ISA's

All add instructions in RISC V and the Motorola 68000

RISC V	Motorola 68000
ADD	ADD
ADD Immediate	ADD Immediate
	Add decimal with extend
	Add Address
	Add Quick
	Add Extend

In RISC, these specialized Instructions have to be emulated using cobinations of other Instructions

Usage percentage of CISC Instructions

Instruction type	Usage percentage
Arithmetic, control flow, calls…	84%
Logic	7%
Floating point	4%

90% of all executed instructions are simple
The other 10% are only seldom used and slow down the processor

Other Differences

Type	RISC	CISC
Instruction size and format	All instructions have the same size and format	Instructions can have different lengths and formats
Registers	Many registers for different uses	Only few registers
Pipelining	Easily doable	Possible, but very complex
Memory Access	Only possible through LOAD and STORE Instructions	Many different Instructions can access Memory
Execution length	All instructions take one cycle to execute	Instructions can take different amounts of time to execute

Summary

- Flow charts are an easy way to visualize programs
- Memory access instructions
- More flow control instructions
- RISC vs. CISC